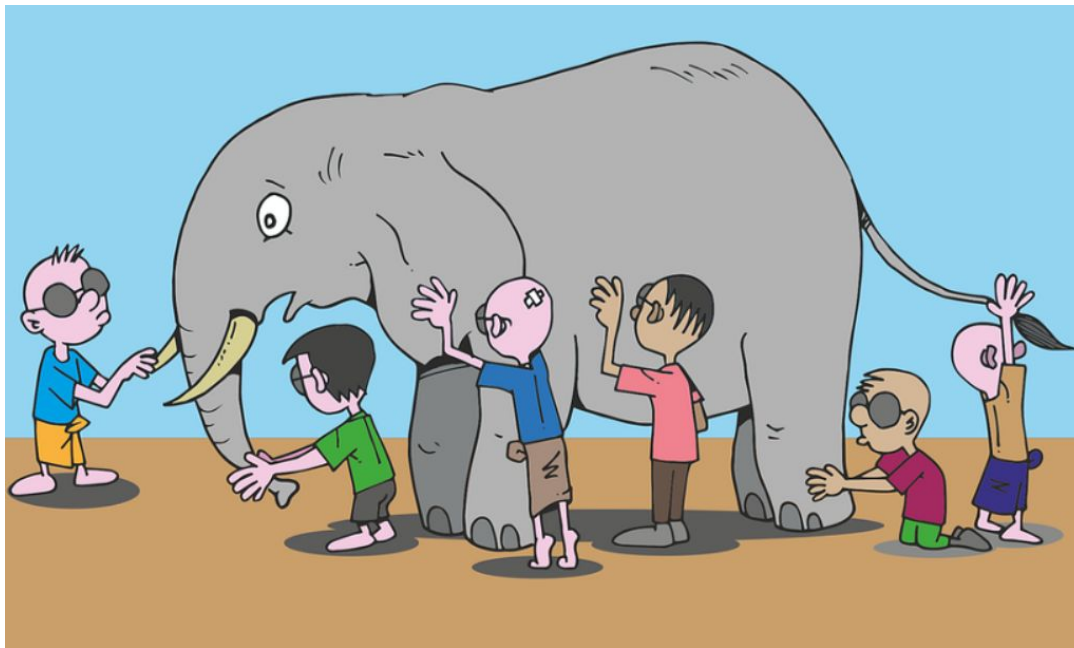


CSE 445: Machine Learning

Ensemble Learning

“The greatest intelligence would not be equal to a comprehension of the whole” - Mikhail Bakunin



What is Ensemble Learning

- Scenario: you need to buy a smartphone. Do you...
 - a) walk into the nearest store and buy the first smartphone the salesperson shows you?
 - b) Extensively research the latest models, check reviews and specs online, consult friends and family, and THEN walk over to buy the phone?
- If you do (b), you're making decisions by an ensemble (i.e. "wisdom of the crowd") – instead of a single factor (i.e. the salesperson)
- Aggregating predictions of a group of predictors (ensemble) are better overall than the best individual predictor
- Ensemble models in machine learning combine the decisions from multiple models to improve the overall performance.

Why use Ensemble Learning?

■ Law of Large Numbers

- Reduces probability for making a wrong prediction, assuming:
 - Classifiers are independent (not true in practice!)
- Suppose you have:
 - n classifiers for a binary classification task
 - Each classifier has same error rate ε
 - Probability mass function (binomial distribution) indicates the probability of error from an ensemble:

$$P(y \geq k) = \sum_k^n \binom{n}{k} \varepsilon^k (1 - \varepsilon)^{n-k} = \varepsilon_{ensemble}$$

- For example if $n = 11$, $\varepsilon = 0.25$, $k = 6$, the probability of error by the ensemble is

Classifier Independence

- Classifiers are not guaranteed to be independent of each other – i.e. errors may be correlated
- However, we can:
 1. Use different algorithms □ every algorithm has its own strengths and weaknesses, having a variety will help achieve lower error rate
 2. Use different features □ different features capture different aspects of a particular class – using a variety helps!
 3. Use different training data □ subject different models to different subsets of training data to reduce variance error

Recall: Bias/Variance Trade-off

ML model's generalization error comes from three sources:

1. Bias $\square$ generalization error due to wrong assumptions
 - e.g. assuming the data is linear when it's actually quadratic
 - High bias will lead to underfitting
2. Variance $\square$ high sensitivity to small variations in training data
 - e.g. polynomial model with many degrees of freedom on quadratic data
 - High variance leads to overfitting
3. Irreducible error $\square$ due to inherent noise of the data itself
 - Make models more complex $\square$ higher variance, lower bias, overfitting
 - Make models less complex $\square$ lower variance, higher bias, underfitting
 - Need to strike a very delicate balance between the two

How to predict with an Ensemble?

- Majority voting
 - Return the most popular prediction from multiple prediction algorithms
- Bootstrap Aggregation – aka Bagging
 - Resample data to train algorithm on different random subsets
- Boosting
 - Reweight data to train algorithm to specialize on “hard” examples – predecessor mistakes
- Stacking
 - Train a model that learns how to aggregate classifiers’ predictions

Majority Voting

- **Hard voting classifier:** aggregate predictions of each classifier, and predict the class that gets the most votes
 - Often achieves higher accuracy than the best classifier in the ensemble
 - Even if each classifier is a weak learner, the ensemble can be a strong learner
 - If trained on the same data, however, their errors are likely to be correlated!
- **Soft voting classifier:** predict the class with the highest class probability, averaged over all individual classifiers
 - More weight to highly confident votes

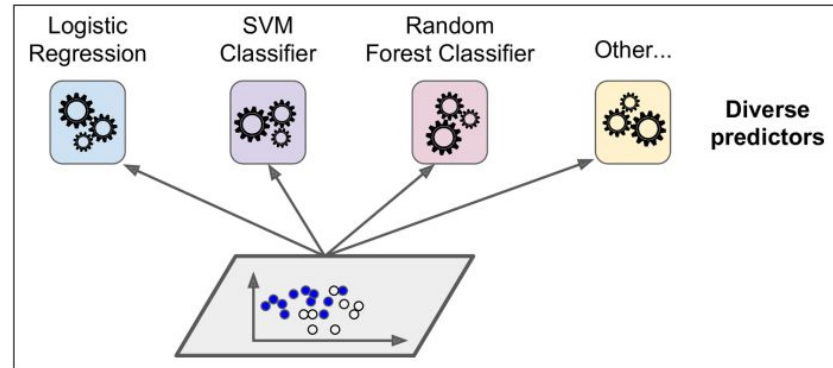


Figure 7-1. Training diverse classifiers

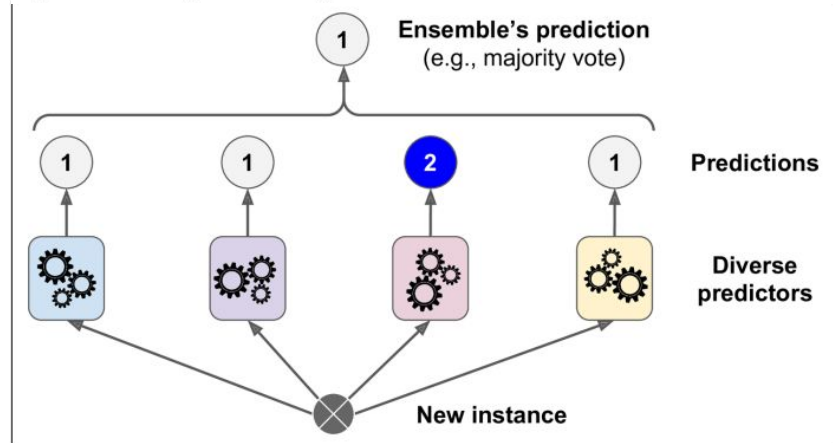


Figure 7-2. Hard voting classifier predictions

Bagging

- Bagging

- Short for Bootstrap AGGregatING
- Use same algorithm on different random subsets of training set
- When sampling is done with replacement, it's bagging
- When sampling is done without replacement, it's pasting
- Prediction made like majority voting

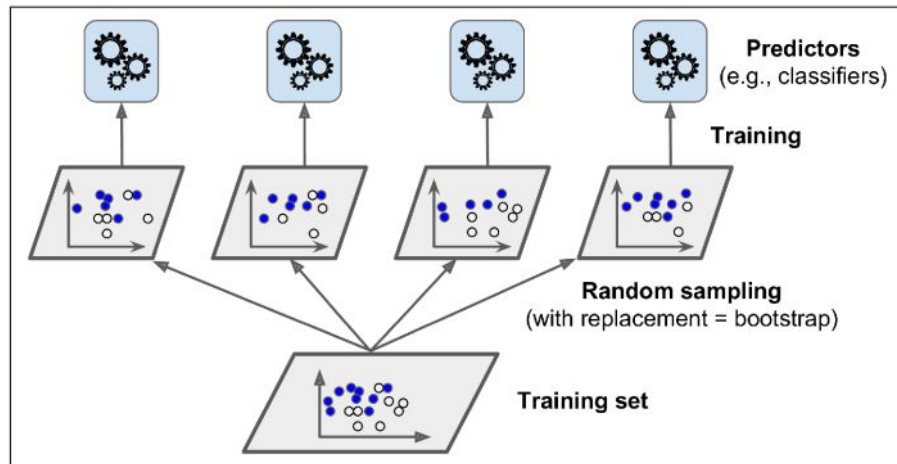


Figure 7-4. Bagging and pasting involves training several predictors on different random samples of the training set

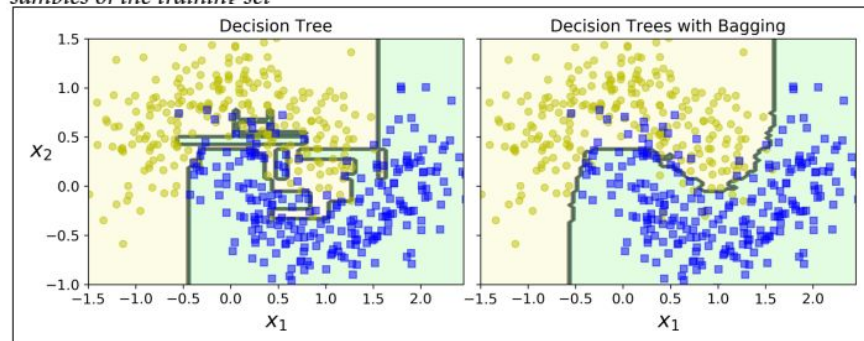


Figure 7-5. A single Decision Tree (left) versus a bagging ensemble of 500 trees (right)

Bagging Example

- Build ensemble from “bootstrap samples” drawn with replacement
- Duplicate data can occur for training for bagging
- No duplicate data in pasting
- Some indices may be missing from training data
- Each classifier is trained on a different subset of data

Sample indices	Bagging Round 1	Bagging Round 2	...	Bagging Round n
1	2	7	...	4
2	2	3	...	3
3	1	2	...	1
4	3	1	...	1
5	7	1	...	2
6	2	7	...	7
7	4	7	...	6

Random Forest

- Random Patches □ sampling both training instances as well as features
- Random Forest □ ensemble of Decision Trees, trained via bagging method
 - Instead of picking very best feature for node splitting, searches for best feature among a random subset of features
 - Higher bias for lower variance
 - Can find out which features are actually important for the model

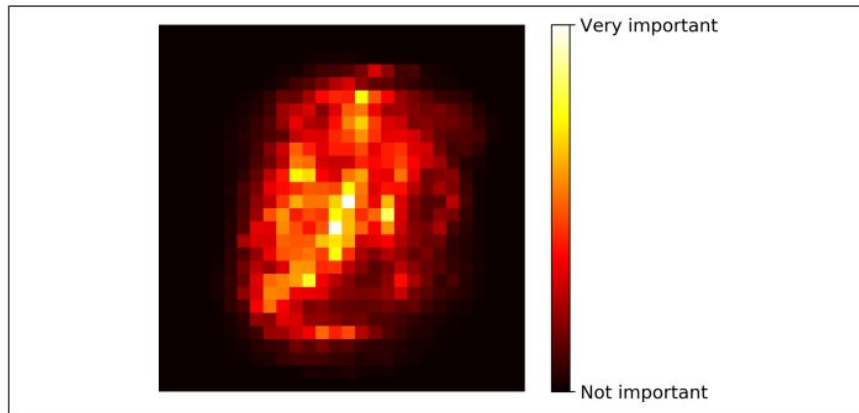


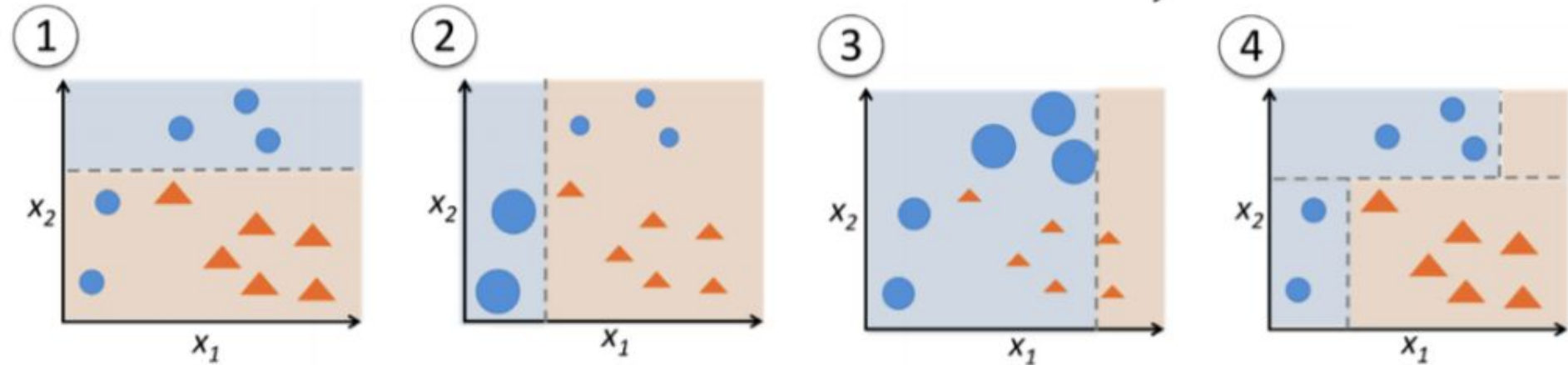
Figure 7-6. MNIST pixel importance (according to a Random Forest classifier)

Boosting – General idea

- Boosting □ sequentially train predictors that each try to predict examples that were hard for previous predictors
- Original Algorithm:
 - Train classifier 1: use random subset of examples without replacement
 - Train classifier 2: use a second random subset of examples without replacement and add 50% of examples misclassified by classifier 1
 - Train classifier 3: use examples that classifiers 1 and 2 disagree on
 - Predict using majority vote from 3 classifiers
- Most popular boosting methods are Adaboost (Adaptive Boosting) and Gradient Boosting

Adaboost

1. Assign equal weights to all examples
2. Assign larger weights to previous misclassifications, and smaller weights to previous correct classifications
3. Assign larger weights to training samples C1 and C2 disagree on, and smaller weights to previously correct classifications
4. Predict with weighted majority vote



Adaboost Example – 1D dataset

Sample indices	x	y	Weights		Correct?	Updated weights
1	1.0	1	0.1	1	Yes	0.072
2	2.0	1	0.1	1	Yes	0.072
3	3.0	1	0.1	1	Yes	0.072
4	4.0	-1	0.1	-1	Yes	0.072
5	5.0	-1	0.1	-1	Yes	0.072
6	6.0	-1	0.1	-1	Yes	0.072
7	7.0	1	0.1	-1	No	0.167
8	8.0	1	0.1	-1	No	0.167
9	9.0	1	0.1	-1	No	0.167
10	10.0	-1	0.1	1	Yes	0.072

Adaboost Example

1. Compute error rate (sum misclassified examples' weights):

$$r_j = 0.1 * 0 + 0.1 * 0 + 0.1 * 0 + 0.1 * 0 + 0.1 * 0 + 0.1 * 0 + 0.1 * 1 + 0.1 * 1 + 0.1 * 1 + 0.1 * 0 = \frac{3}{10} = 0.3$$

2. Compute predictor weight:

$$\alpha_j = \eta \log((1 - r_j)/(r_j)) = 0.5 \log\left(\frac{1 - 0.3}{0.3}\right) = 0.424$$

3. Update weight vector:

$$w = w \exp\left((- \alpha_j)(\hat{y})(y)\right)$$

$$0.1 * \exp(-.424 * 1 * 1) = 0.065 \text{ and } 0.1 * \exp(-.424 * -1 * 1) = 0.153$$

4. Normalize weights to sum to 1:

$$\sum w_i = 7 * 0.065 + 3 * 0.153 = 0.914$$

Sample indices	Correct?	Updated weights
1	Yes	0.072
2	Yes	0.072
3	Yes	0.072
4	Yes	0.072
5	Yes	0.072
6	Yes	0.072
7	No	0.167
8	No	0.167
9	No	0.167
10	Yes	0.072

Stacking

- **Stacked Generalization**
 - Instead of using hard voting to aggregate the predictions of all predictors, why not train a model to perform the aggregation itself?
 - Final predictor (called blender, or meta learner) takes predictions made by each of the ensemble predictors and makes a final prediction

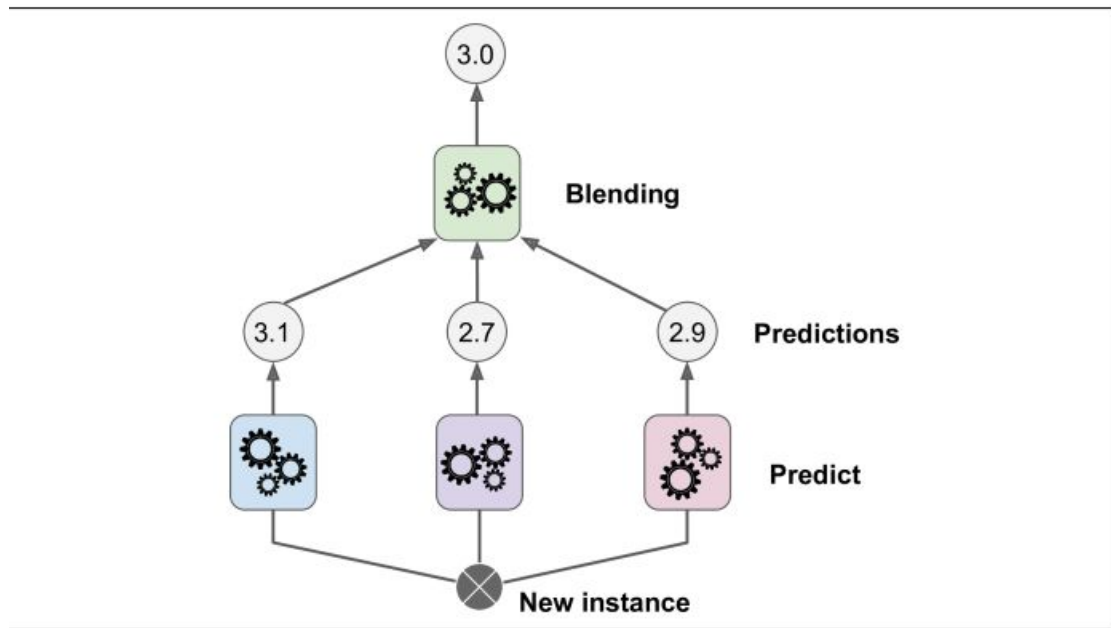


Figure 7-12. Aggregating predictions using a blending predictor

Stacking Algorithm

1. Split dataset into 3 disjoint sets (training, hold-out, test)
2. Train base learners on the first partition
3. Test the base learners on the second partition and third partition
4. Train meta learner on second partition using base learners' predictions as features
5. Evaluate meta learner on third partition using base learners' predictions as features

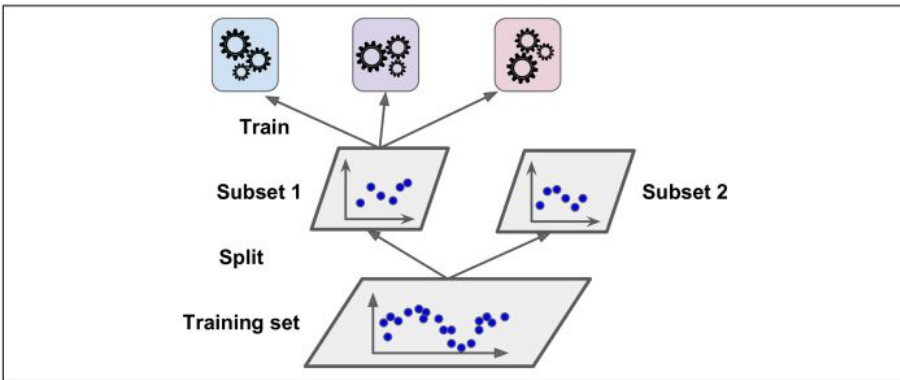


Figure 7-13. Training the first layer

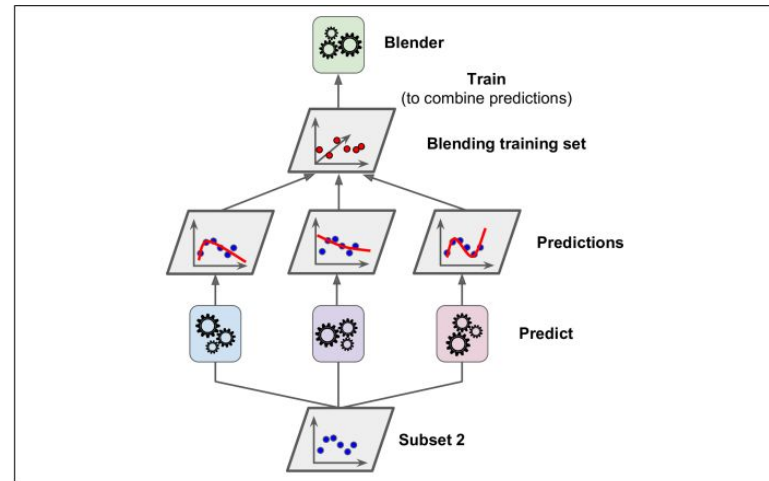


Figure 7-14. Training the blender

Ensemble Learning – Pros and Cons

- Pros:

- Higher accuracy- often the winners of Machine Learning challenges
- With Bagging, you can limit error from variance
- With Boosting, you can limit error from bias
- With Stacking, you can limit variance and bias error while improving accuracy

- Cons:

- Can be harder to interpret
- More difficult to create and train
- More difficult to DEPLOY in production (key reason why it's not widespread in industry)